Queue, Circular Queue, Deque และ Priority Queue

Data Structures & Object-Oriented Programming



Table of Contents

- Queue คืออะไร
- Linked List Queue
- Circular Queue
- Deque และ Priority Queue
- Priority Queue
- implement queues

เป้าหมายการเรียนรู้ของสัปดาห์นี้

- Queue บังคับใช้กฎ FIFO (First In, First Out)
- ตัวแปรพิเศษ 3 แบบที่ต่อยอดจากแนวคิดพื้นฐานของ Queue ได้แก่
  - Circular Queue ที่แก้ปัญหาการสิ้นเปลืองพื้นที่หน่วยความจำเมื่อ implement ด้วย array
  - Deque ที่ยืดหยุ่นให้เข้าถึงได้ทั้งสองด้าน
  - Priority Queue ที่ให้ความสำคัญกับลำดับความสำคัญมากกว่าลำดับการมาถึง
- เมื่อเรียนจบสัปดาห์นี้แล้ว ผู้เรียนควร
  - สามารถอธิบายหลักการทำงานและความแตกต่างของ Queue ทั้งสี่รูปแบบได้อย่างถูกต้อง
  - สามารถ implement Linked List Queue และ Circular Queue ด้วยตนเองได้อย่างสมบูรณ์
  - สามารถวิเคราะห์ได้ว่าสถานการณ์ใดควรเลือกใช้ Queue รูปแบบใด พร้อมอ้างอิง trade-offs ที่เกี่ยวข้องได้อย่างมีเหตุผล

เชื่อมโยงจาก Stack สู่ Queue

- Stack คือ Linked List ที่ถูกจำกัดให้ทำงานที่ปลายด้านเดียวเท่านั้น (head) ทั้งการเพิ่มและการนำออกจึงเกิดขึ้นที่ตำแหน่งเดียวกันเสมอ
  - Stack จึงต้องการ pointer เพียงตัวเดียวคือ _top (ซึ่งก็คือ _head ของ Linked List นั่นเอง)
- Queue มีความแตกต่างตรงที่การเพิ่มข้อมูลและการนำข้อมูลออกเกิดขึ้นกันคนละปลาย
  - ข้อมูลใหม่จะถูกเพิ่มเข้าที่ปลายด้านหนึ่ง (เรียกว่า rear)
  - ข้อมูลถูกนำออกจากปลายอีกด้านหนึ่ง (เรียกว่า front)
  - Queue จึงจำเป็นต้องมี pointer สองตัวพร้อมกันเสมอ คือ
    - _head (สำหรับตำแหน่ง front)
    - _tail (สำหรับตำแหน่ง rear) ตารางต่อไปนี้สรุปความแตกต่างที่สำคัญนี้ไว้อย่างชัดเจน

เชื่อมโยงจาก Stack สู่ Queue

ด้าน	Stack	Queue
หลักการ	LIFO (เข้าหลังออกก่อน)	FIFO (เข้าก่อนออกก่อน)
ตำแหน่งที่เพิ่มข้อมูล	`top` (= `head`)	`rear` (= `tail`)
ตำแหน่งที่นำข้อมูลออก	`top` (= `head`)	`front` (= `head`)
จำนวน pointer ที่ต้องดูแล	1 ตัว (`_top`)	2 ตัว (`_head`, `_tail`)

- แม้ Queue จะต้องดูแล pointer มากกว่า Stack ถึงสองเท่า แต่ทุก operation ของ Queue ก็ยังคงมีความซับซ้อนเป็น O(1)
- เพียงแต่ต้องอาศัยการมี _tail pointer ที่ชี้ไปยัง Node สุดท้ายอยู่เสมอ

ทำไมต้องมี Queue?

- **สถานการณ์ที่ 1 — คิวรอพิมพ์เอกสาร**
  - สมมติว่านิสิต 3 คนส่งงานพิมพ์ไปยังเครื่องพิมพ์ส่วนกลางตามลำดับ ก่อน คือ ก ตามด้วย ข และ ค
  - เมื่อเครื่องพิมพ์ว่างลง คำถามคือใครควรได้รับการพิมพ์ก่อน
  - คำตอบที่สอดคล้องกับสามัญสำนึกคือ ก เพราะเป็นผู้ส่งงานเข้ามาก่อนใคร
    - หากระบบนี้ถูกสร้างขึ้นด้วย Stack แทนที่จะเป็น Queue ผลลัพธ์ที่ได้จะกลายเป็น ค ได้รับการพิมพ์ก่อน
- **สถานการณ์ที่ 2 — ระบบรับ request ของเว็บเซิร์ฟเวอร์**
  - เมื่อเว็บเซิร์ฟเวอร์ได้รับคำขอ (HTTP request) จากผู้ใช้หลายคนพร้อมกัน ระบบควรประมวลผลคำขอตามลำดับที่ได้รับเข้ามา ไม่ใช่ประมวลผลคำขอที่เพิ่งมาถึงล่าสุดก่อน
  - ไม่เช่นนั้นคำขอที่มาถึงก่อนหน้าอาจต้องรอคอยอย่างไม่มีที่สิ้นสุดหากมีคำขอใหม่ทยอยเข้ามาอย่างต่อเนื่อง

ทำไมต้องมี Queue?

- **สถานการณ์ที่ 3 — ระบบคิวธนาคารที่มีลูกค้า VIP**
  - ในระบบคิวธนาคารทั่วไป ลูกค้าจะได้รับบริการตามลำดับการมาถึง
  - แต่หากธนาคารมีบริการพิเศษสำหรับลูกค้า VIP ที่ต้องได้รับบริการก่อนเสมอไม่ว่าจะมาถึงเมื่อใดก็ตาม
    - สถานการณ์นี้ไม่สามารถแก้ไขได้ด้วย Queue แบบธรรมดาอีกต่อไป
    - ต้องอาศัยโครงสร้างข้อมูลชนิดพิเศษที่เรียกว่า Priority Queue

รูปแบบร่วมของทุกสถานการณ์

- ทั้ง 3 สถานการณ์ข้างต้นล้วนต้องการหลักการที่ตรงกันข้ามกับ Stack โดยสิ้นเชิง
  - “สิ่งที่เข้ามาก่อน จะต้องออกไปก่อน”
- ในบางกรณี (ดังเช่นสถานการณ์ที่ 3) ยังต้องการเงื่อนไขเพิ่มเติมว่า “สิ่งที่มีความสำคัญมากกว่า จะต้องออกไปก่อน” โดยไม่คำนึงถึงลำดับเวลาที่เข้ามา

ปัญหาของการใช้ Python List ธรรมดาแทน Queue

- ผู้เรียนที่เพิ่งได้ยินหลักการ FIFO อาจคิดว่าสามารถใช้ Python List ธรรมดาจำลองพฤติกรรมนี้ได้โดยตรง เพียงแค่เพิ่มข้อมูลที่ด้านหลังของ list และนำข้อมูลออกจากด้านหน้า
- ดังตัวอย่างต่อไปนี้

```
queue = []
queue.append("A")    # เพิ่มที่ท้าย
queue.append("B")
queue.append("C")
queue.pop(0)          # นำออกจากหน้า -> "A"
```

- **โค้ดชุดนี้ให้ผลลัพธ์ที่ถูกต้องในเชิงตรรกะทุกประการ แต่กลับมีปัญหาด้านประสิทธิภาพที่ร้ายแรงซ่อนอยู่**
- **นั่นคือ** list.pop(0) **มีความซับซ้อนเป็น** O(n) **ไม่ใช่** O(1)

ปัญหาของการใช้ Python List ธรรมดาแทน Queue

- เหตุผลของความซับซ้อนนี้เชื่อมโยงโดยตรงกับหลักการที่เคยเรียนในสัปดาห์ Dynamic Array ว่า
  - Python List เก็บข้อมูลไว้ในหน่วยความจำที่ต่อเนื่องกัน
  - เมื่อมีการนำ element ที่ตำแหน่ง 0 ออกไป Python จำเป็นต้องเลื่อน element ที่เหลือทั้งหมดไปทางซ้ายหนึ่งตำแหน่ง เพื่อรักษาความต่อเนื่องของหน่วยความจำเอาไว้

```
ก่อน pop(0):   [A, B, C, D, E, ...]
หลัง pop(0):   [B, C, D, E, ...]          ← ทุกตัวเลื่อนซ้ายหนึ่งช่อง = O(n)
```

- **หาก Queue มีขนาดใหญ่มาก เช่นมีสมาชิกอยู่หนึ่งล้านตัว การ dequeue แต่ละครั้งจะต้องเลื่อน element มากถึง 999,999 ครั้ง**
  - **ทั้งที่ในทางตรรกะแล้วเราต้องการเพียงนำ element ตัวแรกออกไปเท่านั้น**
  - **ดังนั้น Python List ไม่ใช่โครงสร้างข้อมูลที่เหมาะสมสำหรับ Queue**
- **Queue ที่ดีควรมีความซับซ้อนเป็น O(1) ทั้ง enqueue และ dequeue ซึ่งสามารถทำได้ด้วยการใช้ Linked List ที่มีทั้ง head และ tail pointer**

Queue คืออะไร

หลักการ FIFO

- Queue เป็นโครงสร้างข้อมูลที่ทำงานตามหลักการที่เรียกว่า FIFO (First In, First Out) ซึ่งแปลตรงตัวได้ว่า “สิ่งที่เข้ามาก่อน จะถูกนำออกก่อน”

การเปรียบเทียบ: คิวซื้อตั๋วภาพยนตร์

- ลองจินตนาการถึงคิวซื้อตั๋วภาพยนตร์ที่พบเห็นได้ทั่วไป
- ผู้ที่มาต่อคิวก่อนย่อมได้รับสิทธิ์ซื้อตั๋วก่อนเสมอ
- ส่วนผู้ที่มาใหม่จะต้องไปต่อท้ายแถวเท่านั้น
- ไม่มีใครสามารถแทรกเข้ากลางคิวได้ (อย่างน้อยก็ในสถานการณ์ที่ทุกคนปฏิบัติตามกติกา)
- ลักษณะดังกล่าวนี้ คือ แก่นแท้ของพฤติกรรมแบบ FIFO

คำศัพท์ที่ใช้เรียกตำแหน่งใน Queue

- ตำแหน่งที่ข้อมูลใหม่ถูกเพิ่มเข้ามาเรียกว่า rear (หรือบางครั้งเรียกว่า back)
- ส่วนตำแหน่งที่ข้อมูลถูกนำออกเรียกว่า front
- ทั้งสองตำแหน่งนี้อยู่กันคนละด้านของ Queue เสมอ
    - ต่างจาก Stack ที่ทั้งการเพิ่มและการนำออกเกิดขึ้นที่ตำแหน่งเดียวกัน (top)

Operations หลักของ Queue

- Queue มี operation หลัก 4 ตัว ซึ่งทุกตัวควรมีความซับซ้อนเป็น O(1) หากถูก implement อย่างเหมาะสม

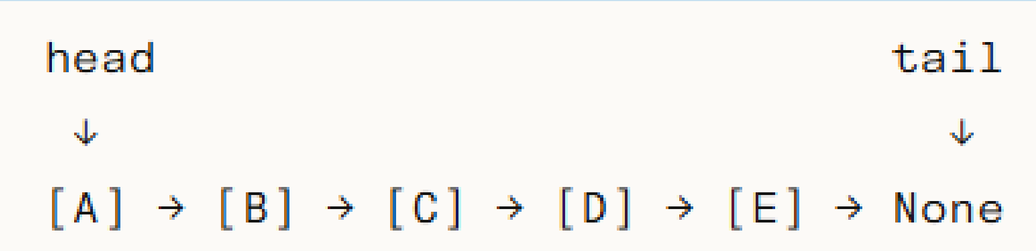
Operation	ความหมาย	Complexity (เมื่อ implement ด้วย Linked List ที่มี tail pointer)
`enqueue(item)`	เพิ่ม `item` เข้าที่ด้านหลัง (rear)	O(1)
`dequeue()`	นำ `item` ที่อยู่ด้านหน้า (front) ออก แล้วคืนค่ากลับมา	O(1)
`peek()`	ดูค่า `item` ที่อยู่ด้านหน้าโดยไม่นำออกจาก Queue	O(1)
`is_empty()`	ตรวจสอบว่า Queue ว่างเปล่าอยู่หรือไม่	O(1)

- **Queue เป็นโครงสร้างข้อมูลเชิงนามธรรม (ADT) ที่จงใจจำกัดความสามารถของตัวเองไว้เพียงเท่านี้**
- **ไม่มี operation ใดที่เข้าถึงข้อมูลตรงกลางหรือค้นหาข้อมูลได้โดยตรง**
  - **เพื่อรับประกันว่าพฤติกรรมของ Queue จะเป็นไปตามหลัก FIFO เสมอ**

Linked List Queue

Linked List Queue

- Queue ต้องทำ enqueue ที่ด้านหลัง (rear) อยู่ตลอดเวลา หากไม่มี _tail pointer การ enqueue แต่ละครั้งจะต้อง traverse จาก _head ไปจนถึง Node สุดท้ายก่อน ซึ่งมีความซับซ้อนเป็น O(n)
  - ยิ่ง Queue มีขนาดใหญ่ขึ้นเท่าใด การ enqueue แต่ละครั้งก็จะยิ่งช้าลงเท่านั้น
- พิจารณา Queue ที่มีทั้ง _head และ _tail ดังภาพต่อไปนี้



การ dequeue ทำได้โดยการนำ Node ที่ _head ชี้อยู่ออกไป แล้วเลื่อน _head ไปยัง Node ถัดไป

- dequeue(): นำ A ออก → head เลื่อนไปที่ B -- O(1)

การ enqueue ทำได้โดยการเชื่อม Node ใหม่เข้ากับ Node ที่ _tail ชี้อยู่ แล้วเลื่อน _tail มาที่ Node ใหม่

- enqueue(F): tail.next = F, tail = F -- O(1)

กรณีพิเศษที่ต้องระวังเป็นพิเศษ

- เมื่อ dequeue จน Queue เหลือ Node เพียงตัวเดียว
- แล้ว dequeue อีกครั้งจนว่างเปล่า ทั้ง _head และ _tail จะต้องกลายเป็น None พร้อมกัน
- หากโค้ดอัปเดตเฉพาะ _head แต่ลืมอัปเดต _tail ให้เป็น None ด้วย จะเกิดสถานการณ์ที่ _tail ยังคงชี้ไปยัง Node ที่ถูกนำออกไปจาก Queue เรียบร้อยแล้ว
- ปัญหานี้จะไม่แสดงอาการใดๆ ในทันที แต่จะไปปรากฏในการ enqueue ครั้งถัดไป เพราะโค้ดจะเขียน self._tail.next = new_node ไปยัง Node เก่าที่ไม่มีอยู่ใน Queue จริงแล้ว
- ทำให้ Node ใหม่ที่เพิ่งเพิ่มเข้าไปไม่ถูกเชื่อมต่อเข้ากับ Queue อย่างถูกต้อง และจะไม่สามารถถูก dequeue ออกมาได้อีกเลย
- ข้อผิดพลาดประเภทนี้อันตรายเป็นพิเศษเพราะไม่ทำให้โปรแกรม crash ทันที แต่จะทำให้ข้อมูล “หายไปเงียบๆ” ซึ่งตรวจพบได้ยากกว่าข้อผิดพลาดที่ทำให้เกิด error ทันที

Circular Queue

ทำไมต้องมี Circular Queue

- ในบางสถานการณ์ที่ทราบขนาดสูงสุดของ Queue ล่วงหน้าอยู่แล้ว และต้องการประหยัดหน่วยความจำโดยไม่ต้องมี overhead จากฟิลด์ next ของแต่ละ Node
  - การ implement Queue ด้วย array ก็เป็นทางเลือกที่น่าสนใจ
- แต่การ implement Queue ด้วย array แบบตรงไปตรงมา (ให้ front เลื่อนไปเรื่อยๆ ทุกครั้งที่ dequeue โดยไม่นำตำแหน่งที่ว่างกลับมาใช้ใหม่) จะประสบปัญหาการสิ้นเปลืองพื้นที่อย่างรุนแรง
- ลองพิจารณาตัวอย่างต่อไปนี้ที่ array มีขนาดคงที่ (capacity) เท่ากับ 5

```
เริ่มต้น (capacity=5):
[A, B, C, D, E]
 ↑front      ↑rear
```

หลังจาก dequeue ไปสามครั้ง

```
            ↓front
[_, _, _, D, E]
               ↑rear
```

หากต้องการ enqueue เพิ่มอีกสองตัวคือ F และ G
[_, _, _, D, E, F, G] ← ต้องขยาย array ทั้งที่ด้านหน้ายังว่างอยู่ถึง 3 ช่อง!

array แบบตรงไปตรงมาไม่มีกลไกใดที่จะนำพื้นที่ว่างเหล่านั้นกลับมาใช้ใหม่ได้เลย
ทำให้ต้องขยายขนาด array อยู่เรื่อยๆ ทั้งที่ในความเป็นจริงมีพื้นที่เหลือเฟือให้ใช้งานอยู่แล้ว

หลักการของ Circular Queue

- Circular Queue (หรือเรียกอีกชื่อหนึ่งว่า Ring Buffer) แก้ปัญหานี้ด้วยแนวคิดที่ว่า
  - เมื่อตำแหน่ง rear เลื่อนไปถึงจุดสิ้นสุดของ array แล้ว ให้ “วนกลับ” มาเริ่มต้นที่ตำแหน่งแรกสุดของ array ใหม่อีกครั้ง แทนที่จะขยายขนาด array ออกไปเรื่อยๆ
- ลองพิจารณาสถานการณ์เดิมจากหัวข้อก่อนหน้า

```
[_, _, _, D, E]
          ↑front
```

แทนที่จะขยาย array ออกไป Circular Queue จะให้ rear วนกลับมาเริ่มที่ตำแหน่ง 0 ใหม่

```
enqueue(F):  rear = (4 + 1) % 5 = 0  →  [F, _, _, D, E]
enqueue(G):  rear = (0 + 1) % 5 = 1  →  [F, G, _, D, E]
```

จะเห็นได้ว่าพื้นที่ว่างสามช่องที่เคยถูกปล่อยทิ้งไว้เปล่าประโยชน์ ถูกนำกลับมาใช้งานใหม่ได้อย่างเต็มประสิทธิภาพ โดยไม่ต้องขยายขนาด array

สูตรหลักของ Circular Queue

- หัวใจสำคัญของ Circular Queue คือ การใช้ modular arithmetic (การหารเอาเศษ) ในการเลื่อนตำแหน่งของ pointer ทั้งสองตัวเสมอ แทนที่จะบวกเพิ่มตรงๆ

```
enqueue:   _rear  = (_rear + 1) % _capacity
dequeue:   _front = (_front + 1) % _capacity
```

- การใช้ % _capacity ทำให้เมื่อค่าตัวเลขเพิ่มขึ้นจนถึง _capacity พอดี ผลลัพธ์ของการหารเอาเศษจะกลับไปเป็น 0 โดยอัตโนมัติ
  - เปรียบเสมือนการ “วนกลับ” มาที่จุดเริ่มต้นของ array
- หลักการนี้เองเป็นที่มาของชื่อ “Circular” เพราะพฤติกรรมของ index ที่วนกลับไปกลับมาเช่นนี้มีลักษณะคล้ายกับการเดินวนบนวงกลม ไม่ใช่การเดินเป็นเส้นตรงเหมือน array ทั่วไป

ปัญหาคลาสสิก: การแยกแยะระหว่าง Full กับ Empty

- Circular Queue มีปัญหาที่พบได้บ่อยในการ implement ครั้งแรก
  - เมื่อใดก็ตามที่ _front และ _rear มีค่าเท่ากันพอดี (_front == _rear) จะไม่สามารถบอกได้อย่างชัดเจนว่า Queue กำลังว่างเปล่าอยู่ หรือกำลังเต็มพอดี
  - เพราะทั้งสองสถานการณ์นี้สามารถทำให้ _front และ _rear มีค่าเท่ากันได้เหมือนกันทุกประการ
    - กรณีว่างเปล่าเพราะไม่เคยมีการ enqueue เลย
    - กรณีเต็ม _rear วนมาบรรจบกับ _front พอดีหลังเพิ่มข้อมูลจนครบ capacity
- วิธีแก้ปัญหาที่ชัดเจนและเข้าใจง่ายที่สุดคือการเพิ่มตัวแปร _size แยกต่างหากออกมา เพื่อนับจำนวน element ที่มีอยู่จริงในขณะนั้น โดยกำหนดว่า
  - หาก _size == 0 แสดงว่า Queue ว่างเปล่า
  - หาก _size == _capacity แสดงว่า Queue เต็ม
- การใช้ _size เป็นตัวตัดสินสถานะแทนการเปรียบเทียบ _front กับ _rear โดยตรง ทำให้ไม่มีความกำกวมเกิดขึ้น

ปัญหาคลาสสิก: การแยกแยะระหว่าง Full กับ Empty

- Circular Queue มีปัญหาที่พบได้บ่อยในการ implement ครั้งแรก
  - เมื่อใดก็ตามที่ _front และ _rear มีค่าเท่ากันพอดี (_front == _rear) จะไม่สามารถบอกได้อย่างชัดเจนว่า Queue กำลังว่างเปล่าอยู่ หรือกำลังเต็มพอดี
  - เพราะทั้งสองสถานการณ์นี้สามารถทำให้ _front และ _rear มีค่าเท่ากันได้เหมือนกันทุกประการ
    - กรณีว่างเปล่าเพราะไม่เคยมีการ enqueue เลย
    - กรณีเต็ม _rear วนมาบรรจบกับ _front พอดีหลังเพิ่มข้อมูลจนครบ capacity
- วิธีแก้ปัญหาที่ชัดเจนและเข้าใจง่ายที่สุดคือการเพิ่มตัวแปร _size แยกต่างหากออกมา เพื่อนับจำนวน element ที่มีอยู่จริงในขณะนั้น โดยกำหนดว่า
  - หาก _size == 0 แสดงว่า Queue ว่างเปล่า
  - หาก _size == _capacity แสดงว่า Queue เต็ม
- การใช้ _size เป็นตัวตัดสินสถานะแทนการเปรียบเทียบ _front กับ _rear โดยตรง ทำให้ไม่มีความกำกวมเกิดขึ้น

Deque และ Priority Queue

หลักการของ Deque

- Deque (ย่อมาจาก Double-Ended Queue) คือ โครงสร้างข้อมูลที่ผ่อนคลายข้อจำกัดของ Queue ธรรมดาลงไปอีกขั้นหนึ่ง
- แทนที่จะจำกัดให้เพิ่มข้อมูลได้เฉพาะด้านหลังและนำออกได้เฉพาะด้านหน้าเท่านั้น Deque อนุญาตให้เพิ่มและนำข้อมูลออกได้จากทั้งสองด้านพร้อมกัน โดยทุก operation ยังคงมีความซับซ้อนเป็น O(1) เช่นเดิม
- ในเชิงโครงสร้างข้อมูลเบื้องหลัง Deque สามารถ implement ได้จาก Doubly Linked List โดยตรง
  - Doubly Linked List มีทั้ง _head และ _tail pointer พร้อมทั้งฟิลด์ prev ที่ทำให้สามารถเพิ่มหรือลบ Node ที่ปลายด้านใดก็ได้ในเวลา O(1) อยู่แล้ว
  - Deque คือ Doubly Linked List ที่เปิดสิทธิ์การเข้าถึงทั้งสองด้านให้ผู้ใช้งานโดยตรง
- ด้วยความยืดหยุ่นนี้ Deque จึงสามารถทำหน้าที่แทนทั้ง Stack และ Queue ได้พร้อมกันเพียงแค่เลือกใช้ method ให้เหมาะสมกับพฤติกรรมที่ต้องการ
  - หากใช้เฉพาะด้านเดียวก็จะได้พฤติกรรมแบบ Stack
  - หากใช้ด้านหนึ่งเพิ่มและอีกด้านหนึ่งนำออกก็จะได้พฤติกรรมแบบ Queue

การใช้งาน collections.deque

- Python มี Deque สำเร็จรูปให้ใช้งานได้ทันทีผ่านโมดูล collections โดยไม่จำเป็นต้อง implement เองตั้งแต่ต้น

```
from collections import deque

dq = deque()
dq.append("B")          # เพิ่มด้านหลัง    -> [B]
dq.appendleft("A")      # เพิ่มด้านหน้า    -> [A, B]
dq.append("C")          # เพิ่มด้านหลัง    -> [A, B, C]

print(dq.pop())          # นำออกด้านหลัง   -> "C", เหลือ deque: [A, B]
print(dq.popleft())      # นำออกด้านหน้า   -> "A", เหลือ deque: [B]
```

สรุป method หลักที่ใช้งานบ่อยที่สุดของ collections.deque

Method	ความหมาย	Complexity
`append(x)`	เพิ่มด้านหลัง	O(1)
`appendleft(x)`	เพิ่มด้านหน้า	O(1)
`pop()`	นำออกด้านหลัง	O(1)
`popleft()`	นำออกด้านหน้า	O(1)
`maxlen=N`	จำกัดขนาดสูงสุด (ลบตัวเก่าสุดออกอัตโนมัติเมื่อเต็ม)	—

กรณีการใช้งานที่สำคัญของ Deque

collections.deque ถูกนำไปใช้งานจริงอยู่บ่อยครั้งใน 3 บริบทหลักดังต่อไปนี้

- **บริบทที่ 1 — Breadth-First Search (BFS)**
  - ในทางปฏิบัตินักพัฒนามักเลือกใช้ deque แทน เพราะมี interface ที่ครบถ้วนกว่าและเป็น built-in ที่ผ่านการปรับแต่งประสิทธิภาพมาอย่างดีแล้ว
- **บริบทที่ 2 — Sliding Window Algorithm**
  - เป็นเทคนิคที่ต้องการเพิ่มข้อมูลใหม่เข้าที่ด้านหลังและนำข้อมูลเก่าที่ไม่ต้องการแล้วออกจากด้านหน้าอยู่ตลอดเวลา (หรือบางครั้งอาจต้องนำออกจากทั้งสองด้านขึ้นอยู่กับเงื่อนไขของปัญหา)
- ตัวอย่างต่อไปนี้แสดงการใช้ deque ที่กำหนด maxlen ไว้ล่วงหน้า เพื่อหาค่ามากที่สุดของแต่ละหน้าต่างขนาด 3 ที่เลื่อนผ่านลำดับตัวเลข

```python
from collections import deque

nums = [1, 3, -1, -3, 5, 3, 6, 7]
window = deque(maxlen=3)

for n in nums:
    window.append(n)
    if len(window) == 3:
        print(max(window), end=" ")

# Output: 3 3 5 5 6 7
```

- การกำหนด maxlen=3 ทำให้ deque จัดการเรื่องการนำข้อมูลเก่าที่เกินขนาดหน้าต่างออกไปให้โดยอัตโนมัติ
- ผู้เขียนโค้ดไม่จำเป็นต้องเขียนเงื่อนไขตรวจสอบและนำออกด้วยตนเอง

กรณีการใช้งานที่สำคัญของ Deque

collections.deque ถูกนำไปใช้งานจริงอยู่บ่อยครั้งใน 3 บริบทหลักดังต่อไปนี้

- **บริบทที่ 3 — ระบบ Undo/Redo ที่มีขนาดจำกัด**
  - ในสัปดาห์ที่แล้วเราเขียนระบบ Undo/Redo ด้วย Stack สองตัว ซึ่งเก็บประวัติได้ไม่จำกัดจำนวน
  - หากต้องการจำกัดจำนวนประวัติที่เก็บไว้สูงสุด (เพื่อประหยัดหน่วยความจำ) การใช้ deque(maxlen=N) แทน Stack ธรรมดาจะช่วยให้ประวัติที่เก่าเกินขอบเขตถูกลบทิ้งโดยอัตโนมัติ
    - โดยไม่ต้องเขียน logic ตรวจสอบขนาดด้วยตนเอง

Priority Queue

หลักการของ Priority Queue

- Priority Queue คือ Queue ชนิดพิเศษที่ทำลายกฎ FIFO แบบดั้งเดิมลงบางส่วน
- โดยกำหนดว่า element ที่มีลำดับความสำคัญ (priority) สูงกว่าจะถูกนำออกก่อนเสมอ ไม่ว่าจะเข้ามาในคิวเมื่อใดก็ตาม
- ในกรณีที่ priority เท่ากันพอดี Priority Queue จึงจะกลับไปใช้หลัก FIFO ตามปกติ (ใครมาก่อนออกก่อน) เป็นตัวตัดสิน
- หลักการนี้ตรงกับสถานการณ์ที่ 4 ของหัวข้อ Motivation พอดี นั่นคือระบบคิวธนาคารที่มีลูกค้า VIP

heapq — เครื่องมือของ Python สำหรับ Priority Queue

- Python ไม่มีคลาส Priority Queue สำเร็จรูปให้ใช้งานตรงๆ
- แต่มีโมดูล heapq ซึ่งเป็นโครงสร้างข้อมูลที่เรียกว่า min-heap
  - มีคุณสมบัติสำคัญคือค่าที่น้อยที่สุดจะถูกดึงออกมาได้ก่อนเสมอ (ตอนนี้ให้เข้าใจเพียงว่า heapq เป็นเครื่องมือสำเร็จรูปที่ทำให้ “ค่าน้อยสุดออกก่อน” ได้อย่างมีประสิทธิภาพ)

```python
import heapq

heap = []
heapq.heappush(heap, 3)
heapq.heappush(heap, 1)
heapq.heappush(heap, 2)

print(heapq.heappop(heap))   # 1 (ค่าน้อยที่สุดออกก่อน)
print(heapq.heappop(heap))   # 2
```

- เนื่องจาก heapq ดึงค่าน้อยที่สุดออกมาก่อนเสมอ
- การนำมาใช้เป็น Priority Queue จึงต้องออกแบบให้ ค่า priority ที่น้อยกว่า หมายถึงความสำคัญที่มากกว่า (เช่น priority = 1 หมายถึงสำคัญที่สุด, priority = 2 หมายถึงสำคัญรองลงมา)

Tuple Pattern — กุญแจสำคัญของการใช้ heapq เป็น Priority Queue

- เมื่อต้องการเก็บทั้งข้อมูลจริง (เช่น ชื่อลูกค้า) และค่า priority ไปพร้อมกันใน heap
- วิธีมาตรฐานที่ใช้กันคือการเก็บเป็น tuple เพราะ Python เปรียบเทียบค่าของ tuple โดยพิจารณาทีละตำแหน่งจากซ้ายไปขวา (หากตำแหน่งแรกเท่ากัน จึงพิจารณาตำแหน่งถัดไป) หลักการนี้ทำให้เราสามารถควบคุมลำดับการเปรียบเทียบได้อย่างแม่นยำ
- รูปแบบมาตรฐานที่ใช้กันคือ (priority, arrival_order, data)
  - arrival_order เป็นตัวนับ (counter) ที่เพิ่มขึ้นทีละหนึ่งทุกครั้งที่มีการเพิ่มข้อมูลใหม่เข้า heap ทำหน้าที่เป็นตัวตัดสินเมื่อ priority เท่ากัน (tiebreaker)
    - เพื่อให้พฤติกรรม FIFO ยังคงทำงานถูกต้องภายในกลุ่มที่มี priority เดียวกัน

Tuple Pattern — กุญแจสำคัญของการใช้ heapq เป็น Priority Queue

```
import heapq

heap = []
counter = 0   # tiebreaker เพื่อรับประกัน FIFO เมื่อ priority เท่ากัน

# priority ยิ่งน้อยยิ่งสำคัญ (1 = VIP, 2 = ทั่วไป)
heapq.heappush(heap, (2, counter, "สมชาย"));    counter += 1
heapq.heappush(heap, (1, counter, "วิภา"));     counter += 1
heapq.heappush(heap, (2, counter, "ประสิทธิ์")); counter += 1

print(heapq.heappop(heap))   # (1, 1, 'วิภา')       <- priority 1 ออกก่อน
print(heapq.heappop(heap))   # (2, 0, 'สมชาย')      <- priority 2, counter เล็กกว่า
print(heapq.heappop(heap))   # (2, 2, 'ประสิทธิ์')
```

- ผลลัพธ์นี้แสดงให้เห็นหลักการทำงานอย่างชัดเจน แม้ “สมชาย” จะเข้าคิวก่อน “วิภา” แต่ “วิภา” กลับได้ออกก่อน เพราะมีค่า priority ต่ำกว่า (สำคัญกว่า)
- ในขณะที่ “สมชาย” กับ “ประสิทธิ์” มี priority เท่ากันพอดี (เท่ากับ 2) จึงต้องพิจารณาตำแหน่งที่สองของ tuple คือ counter ซึ่ง “สมชาย” มีค่าน้อยกว่า (เข้ามาก่อน) จึงถูก pop ออกมาก่อน

เหตุผลที่ต้องมี arrival_order เสมอ

- หากไม่มีตัวตัดสินนี้ และเกิดกรณีที่ priority ของสอง element เท่ากันพอดี Python จะพยายามเปรียบเทียบตำแหน่งถัดไปของ tuple แทน
- หากตำแหน่งนั้นคือข้อมูลจริง (เช่น ชื่อคน หรือ object ที่ซับซ้อนกว่านั้น) อาจทำให้เกิดผลลัพธ์ที่ไม่เป็นไปตาม FIFO ที่คาดหวัง หรือในกรณีเลวร้ายที่สุดอาจทำให้เกิด TypeError ขึ้นได้
- หากข้อมูลนั้นเป็น object ชนิดที่ไม่รองรับการเปรียบเทียบด้วยเครื่องหมาย < (เช่น dictionary หรือ object ที่ไม่ได้ implement __lt__ ไว้)

ความซับซ้อนของ heapq

Operation	Complexity
`heappush`	O(log n)
`heappop`	O(log n)
`heap[0]` (peek ค่าน้อยสุดโดยไม่นำออก)	O(1)

- ข้อสังเกตที่สำคัญคือ heapq มีความซับซ้อนสูงกว่า Queue ธรรมดาที่เคยเรียนมา (O(log n) แทนที่จะเป็น O(1))
- นี่คือต้นทุนที่ต้องแลกเพื่อให้ได้ความสามารถในการจัดลำดับตาม priority ซึ่งเป็นสิ่งที่ Queue ธรรมดาไม่สามารถทำได้เลย
- หลักการนี้สอดคล้องกับแนวคิดเรื่อง trade-offs ที่เน้นย้ำมาตลอดตั้งแต่ต้นวิชา นั่นคือ ความสามารถที่เพิ่มขึ้นมักมาพร้อมกับต้นทุนบางอย่างเสมอ ไม่มีโครงสร้างข้อมูลใดที่ดีที่สุดในทุกด้านพร้อมกัน

คลาส Queue
(Linked List-based)

โค้ดฉบับสมบูรณ์: คลาส Node และ Queue

```python
class Node:
    """Node สำหรับ Linked List-based Queue (เหมือนกับ Node ในสัปดาห์ที่ 10 ทุกประการ)"""

    def __init__(self, data):
        self.data = data
        self.next = None
```

```python
class Queue:
    """
    Queue implementation โดยใช้ Linked List เป็นโครงสร้างเบื้องหลัง
    - enqueue เพิ่มที่ tail (ด้านหลัง / rear)  -> O(1)
    - dequeue นำออกจาก head (ด้านหน้า / front) -> O(1)
    ต้องมีทั้ง _head และ _tail pointer จึงจะได้ O(1) ทั้งคู่
    """

    def __init__(self):
        self._head = None    # ด้านหน้า -- dequeue จากตรงนี้
        self._tail = None    # ด้านหลัง -- enqueue เข้าตรงนี้
        self._size = 0
```

โค้ดฉบับสมบูรณ์: คลาส Node และ Queue

```
def enqueue(self, item) -> None:
    """เพิ่ม item ที่ด้านหลัง (rear) -- O(1)"""
    new_node = Node(item)
    if self._tail is None:              # queue ว่างเปล่าอยู่ก่อนหน้า
        self._head = new_node
        self._tail = new_node
    else:                               # queue ไม่ว่าง
        self._tail.next = new_node      # ขั้นที่ 1: ต่อจาก tail เดิม
        self._tail = new_node           # ขั้นที่ 2: อัปเดต tail
    self._size += 1
```

```
def dequeue(self):
    """
    นำ item ออกจากด้านหน้า (front) และ return -- O(1)
    Raises: IndexError ถ้า queue ว่าง
    """
    if self.is_empty():
        raise IndexError("dequeue from empty queue")
    data = self._head.data
    self._head = self._head.next
    if self._head is None:              # queue ว่างเปล่าลงพอดีหลัง dequeue นี้
        self._tail = None               # ต้อง reset tail ด้วยเสมอ
    self._size -= 1
    return data
```

enqueue แบ่งการทำงานเป็นสองกรณีอย่างชัดเจน

- กรณีแรกคือ Queue ว่างเปล่าอยู่ก่อนหน้า (_tail is None) ซึ่งต้องตั้งค่าทั้ง _head และ _tail ให้ชี้ไปยัง Node ใหม่พร้อมกัน เพราะ Node นี้เป็นทั้ง Node แรกและ Node สุดท้ายในเวลาเดียวกัน
- กรณีที่สองคือ Queue มีข้อมูลอยู่แล้ว เชื่อม Node ใหม่เข้ากับ _tail เดิมก่อน (ขั้นที่ 1) แล้วจึงอัปเดต _tail ให้ชี้มาที่ Node ใหม่ (ขั้นที่ 2)

- dequeue เริ่มต้นด้วยการตรวจสอบ is_empty() ก่อนเสมอ
- เงื่อนไข if self._head is None: self._tail = None ต้องตรวจสอบทุกครั้งว่าการ dequeue ครั้งนี้ทำให้ Queue ว่างเปล่าลงพอดีหรือไม่ หากใช่ ต้องอัปเดต _tail ให้เป็น None ควบคู่ไปด้วยเสมอ

โค้ดฉบับสมบูรณ์: คลาส Node และ Queue

```
def peek(self):
    """
    ดู item ด้านหน้าโดยไม่นำออก -- O(1)
    Raises: IndexError ถ้า queue ว่าง
    """
    if self.is_empty():
        raise IndexError("peek from empty queue")
    return self._head.data

def is_empty(self) -> bool:
    return self._head is None
```

- peek อ่านค่าจาก _head.data โดยตรงโดยไม่เปลี่ยนแปลงโครงสร้างใดๆ
- is_empty ตรวจสอบเพียงว่า _head เป็น None หรือไม่

```
def __len__(self) -> int:
    return self._size
```

```
def __str__(self) -> str:
    if self.is_empty():
        return "Queue: [empty]"
    elements = []
    current = self._head
    while current:
        elements.append(str(current.data))
        current = current.next
    return "FRONT → " + " → ".join(elements) + " → REAR"
```

- __str__ ใช้ traversal pattern พื้นฐานไล่จาก _head ไปยัง _tail เพื่อแสดงผล

คลาส CircularQueue

คลาส CircularQueue

```
class CircularQueue:
    """
    Queue ขนาดคงที่โดยใช้ array และ modular arithmetic
    ใช้ _size เป็นตัวแยกแยะระหว่างสถานะ full กับ empty (ตามหัวข้อที่ 6.4)

    สูตรหลัก:
        enqueue: _rear  = (_rear  + 1) % _capacity
        dequeue: _front = (_front + 1) % _capacity
    """

    def __init__(self, capacity: int):
        self._data = [None] * capacity
        self._capacity = capacity
        self._front = 0      # index ของ element แรก
        self._rear = 0       # index ของตำแหน่งว่างถัดไปที่จะ enqueue
        self._size = 0
```

- __init__ สร้าง array ขนาดคงที่ด้วย [None] * capacity แล้วตั้งค่า _front = 0 และ _rear = 0 ทั้งคู่ เพื่อบอกว่า queue เริ่มต้นว่างเปล่า
- ไม่ได้ใช้ _front == _rear เป็นตัวบอกความว่างเปล่าโดยตรง แต่ใช้ _size == 0 แทน

คลาส CircularQueue

```python
def enqueue(self, item) -> None:
    """
    เพิ่ม item ที่ rear -- O(1)
    Raises: OverflowError ถ้า queue เต็ม
    """
    if self.is_full():
        raise OverflowError("CircularQueue is full")
    self._data[self._rear] = item
    self._rear = (self._rear + 1) % self._capacity    # เลื่อนแบบวนกลับ
    self._size += 1
```

```python
def dequeue(self):
    """
    นำ item ออกจาก front -- O(1)
    Raises: IndexError ถ้า queue ว่าง
    """
    if self.is_empty():
        raise IndexError("dequeue from empty queue")
    data = self._data[self._front]
    self._data[self._front] = None                    # ล้างข้อมูลเก่า (optional)
    self._front = (self._front + 1) % self._capacity  # เลื่อนแบบวนกลับ
    self._size -= 1
    return data
```

- enqueue ตรวจสอบ is_full() ก่อนเสมอ
- หากยังไม่เต็ม จะเขียนข้อมูลลงตำแหน่ง _rear ปัจจุบันก่อน แล้วจึงเลื่อน _rear ด้วยสูตร modular arithmetic
- ลำดับของสองขั้นตอนนี้สำคัญ
  - ต้องเขียนข้อมูลลงตำแหน่งเดิมก่อน แล้วจึงเลื่อน pointer
  - ไม่ใช่เลื่อนก่อนแล้วค่อยเขียน
  - มิฉะนั้นข้อมูลจะถูกเขียนผิดตำแหน่ง

- dequeue ทำงานที่ฝั่ง _front
- บรรทัด self._data[self._front] = None เป็นการล้างข้อมูลเก่าทิ้งหลังจากอ่านค่าออกมาแล้ว
- เป็นแนวปฏิบัติที่ดีเพราะช่วยให้ Python สามารถคืนหน่วยความจำของ object ที่เคยถูกอ้างอิงจากตำแหน่งนั้นได้เร็วขึ้น หากไม่มีตัวแปรอื่นใดอ้างอิงถึง object นั้นอีกแล้ว

คลาส CircularQueue

```
def peek(self):
    """
    ดู item ด้านหน้าโดยไม่นำออก -- O(1)
    Raises: IndexError ถ้า queue ว่าง
    """
    if self.is_empty():
        raise IndexError("peek from empty queue")
    return self._data[self._front]
```

```
def is_empty(self) -> bool:
    return self._size == 0
```

- is_empty และ is_full ทั้งคู่ใช้ _size เป็นตัวตัดสินสถานะโดยตรง
- ไม่มีการเปรียบเทียบ _front กับ _rear เป็นการหลีกเลี่ยงปัญหาความกำกวม

```
def is_full(self) -> bool:
    return self._size == self._capacity

def __len__(self) -> int:
    return self._size
```

```
def __str__(self) -> str:
    if self.is_empty():
        return "CircularQueue: [empty]"
    elements = []
    for i in range(self._size):
        idx = (self._front + i) % self._capacity
        elements.append(str(self._data[idx]))
    return (f"FRONT → " + " → ".join(elements) +
            f" → REAR  (size={self._size}/{self._capacity})")
```

- ต้องใช้ลูป for i in range(self._size) ร่วมกับสูตร idx = (self._front + i) % self._capacity เพื่อคำนวณตำแหน่งจริงใน array ของแต่ละ element ตามลำดับจาก front ไปยัง rear ทีละตัว
- วิธีนี้ทำให้มั่นใจได้ว่าจะแสดงผลเฉพาะ element ที่มีอยู่จริงจำนวน _size ตัวเท่านั้น ไม่ปะปนกับตำแหน่งที่เป็น None ที่อาจหลงเหลืออยู่ใน array จากการ dequeue ก่อนหน้า

คลาส PriorityQueue

คลาส PriorityQueue

```python
import heapq


class PriorityQueue:
    """
    Priority Queue โดยใช้ heapq (min-heap) เป็นโครงสร้างเบื้องหลัง
    Priority น้อย = สำคัญกว่า (ออกก่อน)
    Tuple pattern: (priority, arrival_order, data)
    - arrival_order เป็น tiebreaker เพื่อรับประกัน FIFO เมื่อ priority เท่ากัน
    """

    def __init__(self):
        self._heap = []
        self._counter = 0    # arrival_order -- เพิ่มขึ้นทุกครั้งที่ enqueue

    def enqueue(self, item, priority: int) -> None:
        """เพิ่ม item พร้อม priority -- O(log n)"""
        heapq.heappush(self._heap, (priority, self._counter, item))
        self._counter += 1
```

- __init__ สร้าง _heap เป็น list ว่างเปล่าธรรมดา เพราะ heapq module ทำงานร่วมกับ Python list ทั่วไปโดยตรง
- สิ่งที่ทำให้ list นี้มีคุณสมบัติเป็น heap เป็นวิธีที่เราเข้าถึงมันผ่านฟังก์ชันของ heapq เท่านั้น (หากมีการเข้าถึงหรือแก้ไข _heap โดยตรงโดยไม่ผ่าน heapq.heappush/heapq.heappop คุณสมบัติของ heap อาจเสียหายได้
  - นี่คือเหตุผลสำคัญที่ _heap ถูกตั้งเป็น private attribute)

- enqueue สร้าง tuple ตามรูปแบบ (priority, self._counter, item)
- เพิ่ม _counter ขึ้นหนึ่งทุกครั้งหลังการ push สำเร็จ
- ผู้ใช้งานคลาสนี้จากภายนอกไม่จำเป็นต้องรู้เรื่อง tuple pattern หรือ tiebreaker เลย เพียงแค่เรียก enqueue(item, priority) ตามปกติ ความซับซ้อนของการจัดการ tiebreaker ถูกซ่อนไว้ภายในคลาสทั้งหมด นี่คือตัวอย่างที่ดีของหลักการ encapsulation

คลาส PriorityQueue

```
def dequeue(self):
    """
    นำ item ที่ priority น้อยสุดออก -- O(log n)
    Return: (item, priority)
    Raises: IndexError ถ้า queue ว่าง
    """
    if self.is_empty():
        raise IndexError("dequeue from empty priority queue")
    priority, _, item = heapq.heappop(self._heap)
    return item, priority
```

```
def peek(self):
    """
    ดู item ที่ priority น้อยสุดโดยไม่นำออก -- O(1)
    Raises: IndexError ถ้า queue ว่าง
    """
    if self.is_empty():
        raise IndexError("peek from empty priority queue")
    priority, _, item = self._heap[0]
    return item, priority
```

- dequeue และ peek ทำการ unpack tuple ที่ได้จาก heap ออกมาเป็น priority, _, item
  - ใช้ _ (underscore) แทนชื่อตัวแปรสำหรับ arrival_order เพราะค่านี้มีประโยชน์เฉพาะตอนอยู่ใน heap เพื่อใช้เปรียบเทียบเท่านั้น เมื่อดึงออกมาใช้งานจริงแล้วไม่มีความจำเป็นต้องใช้ค่านี้อีก
  - การใช้ _ เป็นชื่อตัวแปรเป็นธรรมเนียมปฏิบัติที่พบได้ทั่วไปใน Python เพื่อสื่อความหมายว่า “ค่านี้จงใจไม่ใช้งาน”
- ทั้งสอง method คืนค่ากลับเป็น (item, priority) ซึ่งสลับลำดับจากที่เก็บอยู่ใน tuple ภายใน heap ((priority, arrival_order, item))
  - เพราะจากมุมมองของผู้ใช้งานภายนอก การรู้ “ข้อมูล” ก่อน “ระดับความสำคัญ” เป็นลำดับที่เป็นธรรมชาติกว่า
- peek เป็น O(1) มาจากคุณสมบัติของ heap ที่ค่าน้อยที่สุดจะถูกจัดเก็บไว้ที่ตำแหน่ง index 0 ของ list เสมอ
  - การเข้าถึง self._heap[0] จึงเป็นเพียงการเข้าถึง index ธรรมดา
- การ heappop ต้องมีการจัดเรียงโครงสร้างภายในของ heap ใหม่หลังจากนำ root ออกไป (กระบวนการนี้เรียกว่า “heapify”) ซึ่งมีความซับซ้อนเป็น O(log n)

คลาส PriorityQueue

```
def is_empty(self) -> bool:
    return len(self._heap) == 0

def __len__(self) -> int:
    return len(self._heap)
```